

Объединение чатов в новую задачу

Два уровня: установленная инфраструктура проекта и Harness UI

AI-Infrastructure · Codex · Claude Code · Cursor
Состояние реализации на 16 сентября 2026 года

Что получил пользователь

Контекст нескольких разговоров можно продолжить в новой задаче с указанием происхождения информации. Сохраняются доступные тексты, решения и прогресс каждого источника; исходные чаты остаются самостоятельными.

Что сделано

- 1. Автоматическое продолжение в проекте.** Проектные hooks сохраняют доступную видимую историю и передают новой задаче контекст недавних чатов той же рабочей копии и Git-ветки. После установки и включения доверенных hooks отдельная команда сохранения не нужна.
- 2. Явное объединение в Harness UI.** Кнопка Merge chats позволяет выбрать несколько сохранённых разговоров, задать целевой клиент и создать новую задачу с фиксированным набором источников.
- 3. Переносимый документ.** Сохранены режимы context-save / context-load для выжимки, выбранной темы и полного предоставленного экспорта. Они подходят для переноса между машинами или рабочими копиями.

Что означает «сохранить решения и прогресс»

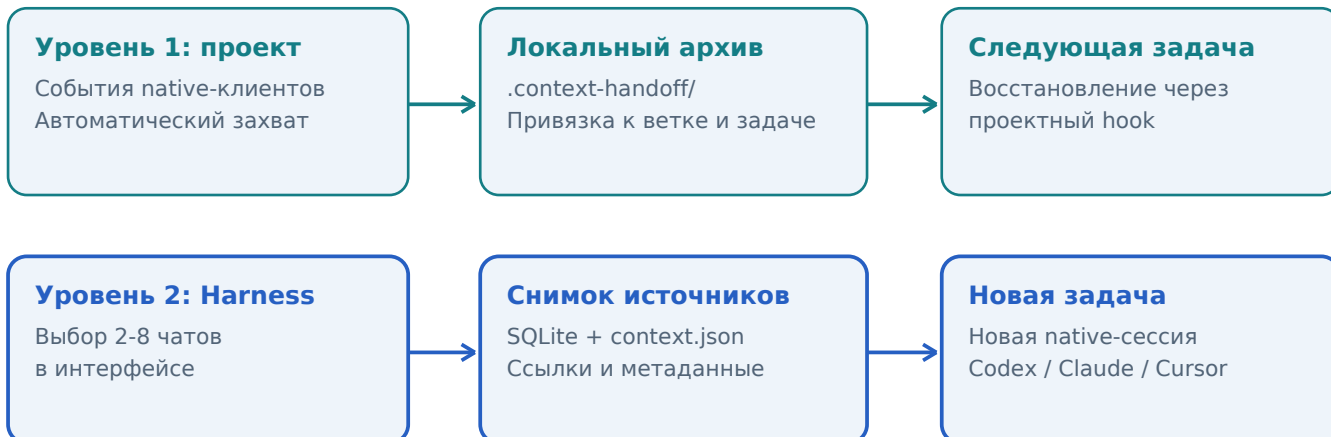
В снимок попадает текст, где они были описаны. Каждый фрагмент сохраняет связь с исходным чатом. Принимающий агент должен сопоставить решения, проверить их по текущему коду и обозначить противоречия. Сам движок хранения не выполняет смысловое согласование и не объявляет конфликтующие решения совместимыми.

Статус поставки

Работа находится в ветке **codex/context-handoff**, в отдельной рабочей копии AI-Infrastructure-context-handoff. Базовые сохранение/загрузка и hooks уже входят в локальные коммиты; переработка объединения и Harness UI на момент подготовки документа остаются незакоммиченными. Коммит, push, PR и развёртывание этих последних изменений не выполнялись.

Документ описывает фактический код и локальную проверку. Наличие функции в этой ветке само по себе не обновляет уже установленную инфраструктуру или запущенный из другой рабочей копии Harness.

Как устроены два уровня



Вопрос	Инфраструктура проекта	Harness UI
Откуда берутся чаты	Из разрешённых событий hooks и явно предоставленного видимого экспорта.	Из сообщений, которые сохранил сам Harness.
Как выбираются источники	Автоматически: до 8 недавних чатов текущей ветки; дополнительно есть явный выбор.	Пользователь выбирает 2-8 неактивных чатов одного зарегистрированного проекта.
Кто создаёт новую задачу	Hook только готовит/доставляет контекст. Создание зависит от инструмента клиента или открытия новой задачи пользователем.	Harness создаёт отдельную запись задачи и запускает новую native-сессию выбранного провайдера.
Граница Git	Та же рабочая копия и та же ветка.	Один проект; ветки источников могут отличаться. Новая задача работает в текущей папке проекта.
Где хранится снимок	.context-handoff/ внутри проекта, игнорируется Git.	Приватное состояние Harness: SQLite и файл архива вне проекта.

Как уровни взаимодействуют

При явном объединении через Harness набор источников уже определён. Поэтому запускаемому клиенту передаётся флаг **CONTEXT_CONTINUITY_RESTORE_DISABLED=1**: проектный hook не добавляет другие недавние чаты. При этом захват нового прогресса продолжает работать.

Хранилища самостоятельны: список Harness не является браузером всех разговоров в Codex Desktop, Claude или Cursor. Установленный runtime не сканирует приватные базы клиентов, а Harness не импортирует автоматически содержимое .context-handoff/.

Автоматическое объединение в установленном проекте

Жизненный цикл без ручного сохранения

Шаг 1. Захват. При отправке запроса и завершении ответа hook передаёт общему Python-runtime разрешённые видимые поля. Если клиент предоставляет подходящий transcript, runtime использует его; иначе сохраняет доступный запрос и ответ.

Шаг 2. Определение области. Runtime связывает данные с рабочей копией, текущей Git-веткой, клиентом и идентификатором разговора. Отсутствие надёжной ветки или идентификатора не заменяется догадкой.

Шаг 3. Новая задача. При SessionStart выбираются до восьми недавних сохранённых чатов текущей ветки, в том числе из разных поддерживаемых клиентов. Из них создаётся отдельный фиксированный архив.

Шаг 4. Доставка. Новая задача получает обзор источников объёмом не более 6 000 UTF-8 байт вместе с заголовками и путь к полному сохранённому архиву. Агенту предписано прочитать опущенные части перед использованием информации.

Шаг 5. Повторное открытие. Восстанавливается тот же набор источников вместе с собственным новым прогрессом задачи. Позднейшие изменения исходных чатов и ротация их снимков не меняют уже подготовленный архив объединения.

Какая информация сопровождает текст

Для источника сохраняются идентификатор разговора и клиент, сведения о коммите, время захвата и отметка полноты. Это позволяет отличать старое решение от текущего состояния проекта и понимать, из какого разговора оно пришло.

Явный выбор вместо автоматического набора

Команда **context-load merge** в Codex или **/context-load merge** в Claude Code и Cursor позволяет перечислить доступные источники и выбрать 2-8 ID. Выбор фиксируется сразу и предназначается следующей новой задаче выбранного клиента.

На сочетание ветки и целевого клиента допускается один ожидающий набор. Повтор с тем же набором допустим, другой запрос не перезаписывает ожидание. Новая задача забирает подготовленный набор под общей блокировкой; исходный возобновляемый чат его не расходует.

Практическая граница автоматизации

Автоматический выбор основан на рабочей копии и ветке, а не на смысловой близости тем. Если в одной ветке обсуждались разные задачи, используйте явный выбор. Hook не открывает окно нового чата самостоятельно.

Клиенты и точность истории

Клиент	Захват	Восстановление
Codex	UserPromptSubmit и Stop	SessionStart; нужен session_id.
Claude Code	UserPromptSubmit и Stop	SessionStart; нужен session_id.
Cursor	beforeSubmitPrompt и afterAgentResponse	sessionStart; нужен conversation_id. Передача additional_context асинхронная.

Это контракты адаптеров, подключённых в репозитории. Их фактическая доставка зависит от версии клиента, включённых hooks, доверия к проекту и доступных событий. Cursor Cloud без sessionStart не может автоматически получить такой контекст. [1, 3]

Четыре разных понятия полноты

Видимый экспорт. Явно предоставленный файл .txt, .md или .markdown с видимой перепиской. Сохраняется реально полученный текст; отсутствие экспорта не восполняется реконструкцией.

Проекция JSONL. Из известных форматов извлекается только видимый текст user и assistant. Системные сообщения, скрытые рассуждения, вызовы инструментов и служебные оболочки отбрасываются. Такой снимок явно считается частичным.

История событий. Когда transcript недоступен, остаются только запросы и ответы, пришедшие в hooks. Задержанные или пропущенные клиентом события не становятся частью архива автоматически.

Обзор для старта. Короткий текст в начальном контексте намеренно ограничен. Полный сохранённый архив может содержать больше, чем этот обзор. Наличие архива не доказывает полноту всей переписки в клиенте.

Доверие и мягкий отказ

Hooks работают через штатное доверие и разрешения клиента. При недоступном runtime, некорректном вводе, превышении лимита, отсутствии нужной идентичности или распознанном вероятном секрете обработчик пропускает операцию и не блокирует текущий ответ клиента. Явные команды list/merge возвращают ошибку, чтобы агент мог её объяснить.

Хранилище не получает доступ к приватным каталогам аккаунтов и внутренним базам переписки. Исторические команды, инструкции и согласия считаются данными, а не полномочиями для новых действий.

Отключение

CONTEXT_CONTINUITY_DISABLED=1 отключает и захват, и восстановление.

CONTEXT_CONTINUITY_RESTORE_DISABLED=1 отключает только восстановление. Оба флага также принимают true и yes.

Выжимка, тема и полный экспорт

Ручной перенос остаётся полезным, когда нужен файл для другой машины, рабочей копии, клиента или проверки человеком. Он дополняет автоматический уровень, а не требуется при каждом завершении чата. [1, 4]

Режим	Содержимое	Лимит
summary	Цель, ограничения, решения, прогресс, проверки, вопросы, следующие шаги и файлы.	8 000 Unicode-символов подготовленного текста.
topic	Те же разделы только по названной теме.	16 000 символов.
full	Подготовленный контекст плюс явно предоставленный видимый экспорт переписки.	64 000 символов подготовленного текста + экспорт до 4 MiB.

Почему full требует файл

В этой задаче «полный контекст» определён как дословная переписка там, где клиент позволяет её экспортировать. Поэтому full требует **--transcript /path/to/export**. Без файла этот режим не выполняется; summary и topic остаются доступны. У Codex не предполагается наличие универсальной команды экспорта. Для Claude и Cursor используется видимый экспорт, если он доступен в конкретном клиенте.

Как использовать

Codex: context-save summary

Claude / Cursor: /context-save summary

Codex: context-load tasks/TASK-001/context-save-<timestamp>.md

Claude / Cursor: /context-load tasks/TASK-001/context-save-<timestamp>.md

Это вызовы навыков или команд агента, а не исполняемые команды shell. Агент готовит документ в едином Markdown-формате, сообщает абсолютный путь и текст для продолжения в следующей задаче.

Что защищает перенос

Загрузка проверяет формат, контрольные суммы и изменение связанных исходных файлов. Дословный transcript включается при загрузке только с явным --include-transcript. Уже существующий выходной файл не перезаписывается. Данные не продвигаются автоматически в Project Brain, Memory Bank или SQLite-индекс.

Сохранение не выполняет git add, commit, push или публикацию. Экспорт ограничен разрешённым видимым текстом: скрытые рассуждения и внутренние журналы клиента в него не подставляются.

Где искать результат

По умолчанию: tasks/TASK-NNN/context-save-<timestamp>.md. Такой документ исключён из автоматического индексирования; его происхождение и актуальность проверяются при загрузке.

Объединение через Harness UI

Пошаговый сценарий

- 1. Открыть Sessions → Merge chats.** Выбрать текущий зарегистрированный проект. Интерфейс покажет подходящие сохранённые чаты этого проекта.
- 2. Выбрать 2-8 источников.** Источники могут принадлежать разным провайдерам. В списке видны название, клиент и состояние; при наличии показывается ветка. Продолжение недоступно, пока выбран неподходящий размер набора.
- 3. Нажать Continue with selected chats.** Появится форма новой задачи с карточками выбранных источников и начальным запросом о сохранении решений, прогресса, противоречий и следующих шагов.
- 4. Выбрать назначение.** Указать Codex, Claude или Cursor, модель, доступную глубину рассуждения и режим Plan/Edit. Можно уточнить запрос. Сценарий фиксирован как обычная задача в текущей папке проекта.
- 5. Нажать Merge into new task.** Сервер проверит источники, сохранит их снимки и создаст отдельную задачу с новой native-сессией. Исходные сессии остаются без изменений.
- 6. Проверить источники.** Кнопка View saved context показывает сохранённые сообщения по каждому чату. Карточки позволяют открыть оригиналы; более позднее изменение оригинала не меняет снимок в новой задаче.

Что можно и нельзя выбрать

Источники должны быть неактивными: queued, running и awaiting_context исключены. Нельзя выбрать Creator, Fleet или Clash. Целевая merge-задача не создаёт новый Git worktree и не связывается с Brain/Clash; вложения в момент объединения не добавляются. Ветки исходных чатов могут отличаться, поэтому агент должен проверять применимость старых решений.

После перезапуска Harness

Если объединённая задача завершилась с ошибкой или была прервана до получения native ID, доступна **Restart merged task**. Она повторно выполняет исходный запрос с сохранёнными настройками и теми же источниками. Если предыдущая попытка успела изменить файлы, это нужно учитывать. При наличии native ID используется обычное продолжение.

Пример

Чат А в Codex содержит решение по API, чат В в Claude - прогресс интерфейса, чат С в Cursor - результаты исследования. Их можно объединить в новую задачу Cursor. В ней сохраняются три самостоятельных источника, а не одна неподписанная смесь текста.

Что реализовано в сервере Harness

Снимок и происхождение

Модуль `chat_merge.py` собирает разрешённые `user/text/result`-события из истории Harness. Ошибочные `result`-события и повтор уже записанного финального ответа отфильтровываются. Для каждого источника сохраняются ID, название, провайдер, ветка, тип `workspace`, временные отметки, статус, граница истории по `event ID` и `SHA-256`. Если источник сам получен объединением, его вложенный контекст также сохраняется.

Два представления одного снимка

Полный JSON хранится в таблице **`session_merges`** в SQLite и в **`state_dir/merges/<session-id>/context.json`**. Обычная выдача списка задач содержит краткие метаданные; полный архив загружается отдельно. Папки создаются приватными, файлы - с правами `0600`. Перед первым запуском файл сверяется с сохранённой копией БД.

Первый запуск и следующие сообщения

Пока `native ID` отсутствует, в запрос добавляется сбалансированный обзор источников до 16 000 UTF-8 байт и путь к полному архиву. После получения `native ID` этот начальный блок не вставляется повторно при обычном продолжении. Для Claude передаётся `--add-dir` только для папки данного архива; остальные клиенты используют свои штатные инструменты чтения файлов.

Повторы и частичные сбои

Запрос объединения содержит `request_id` и отпечаток параметров. Одновременные одинаковые запросы получают одну задачу; изменение параметров с тем же ID отклоняется. Снимок собирается под блокировкой. Запись задачи, связи с архивом и первого сообщения проходит в одной транзакции; при ошибке файл очищается.

После сбоя очереди задача остаётся явно прерванной. Автоматический повтор запроса возобновляет её только если запуск ещё не начинался. После начатой попытки повторный запуск возможен явным действием пользователя. При старте сервер удаляет только принадлежащие ему `UUID`-каталоги архивов без записи в БД; неизвестные файлы и символьные ссылки сохраняются.

Метод	Назначение
POST <code>/api/sessions/merge</code>	Выбранные <code>source_ids</code> + <code>request_id</code> + <code>destination</code> ; создать новую задачу.
GET <code>/api/sessions/<id>/merge</code>	Получить полный сохранённый снимок выбранных источников.
POST <code>/api/sessions/<id>/restart-merge</code>	Явно повторно запустить допустимую прерванную <code>merge</code> -задачу.

Используются существующие границы HTTP API: проверка локального происхождения запросов и токен для изменяющих операций. Идентификаторы источников валидируются; текст в UI вставляется как текст, без исполнения HTML. [5, 6]

Лимиты, приватность и взаимодействие

Ограничение	Проектный runtime	Harness
Источники	До 8 при автоматическом выборе; 2-8 при явном.	2-8 выбранных чатов.
Текст одного источника	До 4 MiB видимого текста; обрезание старого содержимого помечается.	До 512 KiB сообщений; не более 10 000 событий.
Начальный обзор	6 000 UTF-8 байт вместе с заголовками.	16 000 UTF-8 байт.
Один полный архив	До 32 MiB сериализованного JSON.	До 2 MiB сериализованного JSON.
Все архивы	До 128 архивов и 256 MiB на checkout.	До 128 архивов и 64 MiB сериализованной истории в БД; есть и файловые копии.
Новый запрос	Лимиты входного hook-payload и текста runtime.	До 16 KiB для запроса новой merge-задачи.

KiB = 1 024 байта; MiB = 1 048 576 байт. Это не токены модели. JSON-экранирование и метаданные входят в лимиты архива, поэтому суммарный архив может превысить лимит даже при допустимом размере каждого источника.

Общие правила

История не переносит старые разрешения и не запускает встроенные команды. Текущие правила проекта, реальный код и новый запрос пользователя имеют приоритет. Противоречия помечены как not-evaluated до проверки агентом. Ни один механизм не переносит изменения исходного кода между ветками: он передаёт контекст, а не выполняет Git merge.

Различия, которые важно сохранить

Проектный runtime отклоняет распознанные вероятные секреты, отделён от Project Brain/Memory Bank и индекса SQLite. **Harness** сохраняет выбранные видимые сообщения в своей БД; отдельного сканера секретов в chat_merge.py нет. Видимый секрет, уже написанный в чате Harness, может попасть в выбранный снимок.

Архивы локальны, но их текст и обзор используются в запросе к выбранному провайдеру при запуске модели. Приватное хранение на диске не означает, что содержание никогда не передаётся этому провайдеру.

Ограничения файлового доступа

Чтение полного архива зависит от разрешений клиента. Если оно невозможно, агенту предписано сообщить об этом. Доступ Claude к папке архива в Edit-режиме не является защитой от записи: изменение файла приведёт к отказу последующей проверки целостности, а сохранённая копия в БД останется доступной в UI.

Что проверено и что не подтверждено

Проверка	Фактический результат
Общий набор Harness	242 теста: 231 прошёл, 11 пропущены из-за отсутствующего опционального Fleet/LangGraph runtime. Лог: harness-all.log.
Финальные merge + launch регрессии	15 тестов прошли: 14 проверок объединения и одна проверка запуска с тестовыми процессами всех 3 провайдеров. Лог: merge-launch-final.log.
Автоматическая continuity	31 тест прошёл в PHP Core. Runtime и тесты побайтно совпадают в четырёх готовых редакциях. Лог: continuity.log.
Контракт генератора	7 тестов прошли. Лог: runtime-contract.log.
Репозиторные проверки	Parity генераторных assets, зеркала, policy-lock, context budgets, markdown-ссылки, инвентари установки, changelog и git diff --check прошли.
JavaScript	Извлечённый скрипт Harness прошёл node --check.

Результаты относятся к локальным проверкам 16.09.2026. Часть наборов пересекается: числа в таблице не следует складывать как независимые тесты. Общий набор был запущен до добавления последней регрессии автоматического повтора; она затем прошла в отдельном финальном наборе из 15 тестов.

Сценарий во встроенном браузере Codex

На изолированном стенде выбраны чаты Codex, Claude и Cursor. Создана новая задача Cursor; оригиналы остались в списке. View saved context показал сообщения, решения и прогресс каждого источника. После перезапуска сервера та же задача получила статус Interrupted, сохранила источники и была повторно поставлена в очередь через Restart merged task.

Какие ошибки покрывает проверка

В том числе: неверные и повторяющиеся ID, другой проект, активный источник, превышение лимитов, вложенные объединения, Unicode в обзоре, параллельный повтор запроса, откат транзакции, отказ очереди, повреждение архива и очистка файлов после аварийного окна записи. Для провайдеров проверены запуск без прежнего native ID, чтение снимка и сохранение захвата при отключённом restore.

Честные границы результата

Реальные платные вызовы моделей и доставка всех событий hooks из живых аккаунтов не проверялись. Тестовые CLI подтверждают контракт запуска, а не качество смыслового объединения моделью. Проверка выполнялась локально; зелёный удалённый CI и развёртывание не заявляются.

Над Harness работали 10 специалистов и координатор, один автор изменений. Итоговая API/UI-проверка и проверка хранения были независимыми. Длительность этапа Harness составила около 23 минут.

Карта изменений и применение

Область	Что изменено или используется
Общий runtime	memory-bank/scripts/context_continuity.py: захват, автоматический/выбранный merge, фиксированные архивы, восстановление, квоты и restore-only флаг.
Переносимый handoff	memory-bank/scripts/context_handoff.py и context.py; навыки context-save / context-load. Формат, выжимки, экспорт, контроль целостности и актуальности.
Интеграции клиентов	Конфигурации и адаптеры .claude, .cursor, .codex и .agents; синхронизированные навыки, команды, инструкции и hooks.
Готовые редакции	Laravel, Symfony, PHP Core и WordPress. Общий runtime и применимые изменения синхронизированы.
Infrastructure-Creator	Assets memory-seed, runtime-contract, инструкции hook-forge / skill-forge, проверки контрактов и сгенерированные зеркала.
Harness	Новый chat_merge.py; изменения sessions.py, web.py и web/index.html. Тесты merge, sessions и web; документация и CHANGELOG.

Как применить результат

В проекте: установить или обновить соответствующую редакцию инфраструктуры и штатно разрешить hooks в используемом клиенте. Обновление существующего проекта должно сохранить командные изменения; следовать процедуре installer/Infrastructure-Creator. Уже установленная старая версия не меняется сама.

В Harness: запускать сервер из рабочей копии, содержащей эту реализацию. После обновления перезапустить сервер и открыть Sessions → Merge chats. Источники должны уже присутствовать в его истории. Отдельная установка native-hooks в целевом проекте не нужна для самого UI-объединения, но нужна для проектного автоматического уровня.

Исходники и свидетельства

Все пути ниже относительно корню рабочей копии:

[1] docs/CONTEXT-HANDOFF.md

[2] harness/README.md, раздел Merge chats into a new task

[3] PHP Core/memory-bank/scripts/context_continuity.py и конфигурации hooks редакций

[4] PHP Core/memory-bank/scripts/context_handoff.py

[5] harness/src/harness/chat_merge.py, sessions.py, web.py

[6] harness/web/index.html

[7] tests/test_harness_merge.py, test_harness_sessions.py, test_harness_web.py

[8] PHP Core/memory-bank/tests/test_context_continuity.py; Infrastructure-Creator/tests/test_runtime_contract.py

Логи локального этапа Harness: /tmp/harness-merge-validation/. Это временные свидетельства на машине, а не опубликованный CI-отчёт. Документ подготовлен по состоянию кода и сохранённым логам; при последующих изменениях ветки лимиты и поведение следует перепроверить.